



Универзитет „Св. Кирил и Методиј“ во Скопје
Факултет за информатички науки и компјутерско инженерство

ДИПЛОМСКА РАБОТА

Препознавање на AI-генериран програмски код

Ментор
Проф. д-р Соња Гиевска

Кандидат
Марино Јакимоски
211017

Септември 2025

Апстракт

Во денешно време сè поголем дел од програмскиот код е генериран со помош на големи јазични модели. Тоа претставува проблем кога е потребно да се докаже или открие дали еден примерок код е напишан од човек или јазичен модел. Таков пример се академските средини каде што е потребно објективна евалуација на студентите, кои своите задачи може да ги решаваат користејќи јазични модели. Овој дипломски труд има за цел да разгледа различни методи за идентификација на авторството на примероци изворен код, и тоа во две класи: *human* и *ai*. Евалуираните методи се од повеќе различни домени; некои од нив се базирани на класични методи од областа на машинското учење, други на анализа на конкретни синтаксни дрва (парсирачки дрва), или токен базирана класификација користејќи конволуциски и целосно поврзани мрежи. Истражувањата во овој труд покажаа дека на избраното податочно множество со класификација користејќи конволуциски мрежи се добива прецизност и до 98%, додека методите со граф-невронски мрежи со кои се класифицираат синтаксни дрва постигнуваат прецизност до 93%.

Клучни зборови: *синтаксни дрва, граф-невронски мрежи, конволуциски мрежи*

Содржина

Апстракт	ii
1 Вовед	1
2 Сродни истражувања	1
3 Методологија	3
3.1 Експериментална конфигурација	3
3.2 Податоци и нивна припрема	3
3.2.1 Креирање карактеристики	4
3.2.1.a Креирање синтаксни дрва	4
3.2.1.б Репрезентација на податоците за традиционалните ме- тоди од МУ	5
3.2.1.в Креирање графови од синтаксни дрва	5
3.2.1.в.1 Видови граф-базирани множества	6
3.3 Методи	7
3.3.1 Традиционално машинско учење	7
3.3.2 Длабоко учење	7
3.3.2.a Токен базирани методи	7
3.3.2.a.1 CodeBERT Embeddings classification	7
3.3.2.a.2 Multi-Scale CNN + Bi-LSTM	7
3.3.2.a.3 Само Multi-Scale CNN	9
3.3.2.б Граф-базирани методи	9
3.3.2.б.1 Граф-конволуциска мрежа	9
3.3.2.б.2 Graph-attention Мрежа	10
3.3.2.б.3 Graph Transformer Мрежа	10
4 Резултати	11
4.1 Перформанси на традиционалните методи од машинско учење	11
4.2 Перформанси на токен-базираните методи за длабоко учење	12
4.3 Перформанси на граф-базираните методи	13
4.4 Споредбена анализа	14
5 Идни истражувања	15
6 Заклучок	16
7 Библиографија	16

1 Вовед

Со подемот на големите јазични модели, освен тоа што се олесни пишувањето разни текстови: блогови, статии, вести, и слично, се појави и олеснување и во мноштво задачи поврзани со програмски код [1]. Задачите поврзани со пишување програмски код може да се поделат на повеќе видови, од кои најклучни се модификација, исправка или наоѓање грешки, комплетирање на код, и секако генерирање код од промптови напишани со природен јазик [1]. Робустноста на големите јазични модели го прави програмирањето подостапно, полесно и побрзо, дури и за луѓе кои имаат мало или воопшто немаат познавање во областа. [2]

Способноста на големите јазични модели да генерираат код од промптови кои вклучуваат едноставни реченици напишани во природен јазик, или пак цели списоци од функционални барања, е од интерес за луѓе со најразлична позадина: од искусни програмери кои работат на клучни проблеми сè до студенти кои штотуку започнале со учење.

Ова претставува проблем во средини во кои е потребно или обврзано секаков код да биде напишан од човек, или средини во кои е клучно да се избегне плагијаризмот или повреда на авторски права. Еден пример за ваква околина е академската, во која треба објективна евалуација на студентите на техничките факултети. Лесната достапност на големите јазични модели го прави овој проблем прилично голем. Студентите можат само со копирање на текстот од зададените лабораториски задачи да добијат точно решение на истите, со што избегнуваат самостојно решавање на задачата [2]. Со ова го намалуваат својот академски интегритет, вршат еден вид „инфлација“ на крајните оценки кои ќе ги добијат, и дури може да ја намалат вредноста на поседување факултетска диплома.

Во овој дипломски труд вршиме компаративна евалуација врз benchmark податочно множество на повеќе различни пристапи за класификација на програмски код во класите *human* и *ai*. Сепак, систематски споредби на овие различни пристапи се сè уште ограничени. Во рамките на трудот се разгледуваат модели од машинското учење базирани на разни карактеристики, граф-невронски мрежи [3], како и метод на длабоко учење со CNN+BiLSTM[4] архитектура применета на претренирани векторски репрезентации на код (анг: code embeddings). Целта е да се добие увид во ефикасноста и ограничувањата на методите.

2 Сродни истражувања

Во овој дел се врши презентација и анализа на постоечки методи за класификација на програмски код кои се од сродна природа со методите опишани во овој труд.

Честа методологија за класификација на код е користење синтаксни дрва. Во [5] се разгледуваат различни методи за репрезентација на код преку апстрактни синтаксни дрва. Доаѓаат до заклучок дека модели кои се тренирани на синтаксните дрва даваат конзистентно полоши резултати од модели тренирани врз токенизирана репрезентација при детекција на код-клонови, сумаризација на код и пребарување. Од друга страна, бидејќи синтаксните дрва може да бидат многу големи и длабоки, [6] предлага метод за разделување на дрвата во мали поддрва за секоја наредба во кодот. Секое поддрво потоа се енкодира во векторска репрезентација, и рекурентна неврон-

ска мрежа ги енкодира овие векторски низи од секое поддрво во конечна векторска репрезентација на целиот код. Со овој го решаваат проблемот за клон детекција и класификација на изворен код подобро отколку дотогашните најдобри методи.

Друга методологија во анализа и класификација на код со користење апстрактни синтаксни дрва е збогатување на истите со дополнителни информации - јазли и ребра од различни типови. На пример, во [7] имплементираат метода за репрезентација на *intermediate code* како граф, кој потоа го збогатуваат со ребра контролниот и податочниот тек низ кодот, кои потоа ги класифицираат користејќи софтверски класификатор. Освен ребра за контролен и податочен тек, во [8] графовите ги дополнуваат со ребра како „nextSib“, „nextToken“, „nextUse“, но додаваат и ребра со кои го репрезентираат контролниот тек. Имено, кај појавата на јазол за *if*-израз во синтаксното дрво, додаваат ребра со кои почетниот јазол го поврзуваат со јазолот за телото на изразот („condTrue“, *condition true*), или со јазолот за *else*-израз во случај на „condFalse“. Со ова во апстрактните синтаксни дрва се добива подетална репрезентација на структурата и семантиката на кодот, што им помага на граф-невронските мрежи подобро да ги научат карактеристиките на кодот. Збогатени апстрактни синтаксни дрва се користат и во [9]. Тука предлагаат комбинирање на синтаксните дрва со ребра за контролен и податочен тек, и потоа конвертирање на добиените дрва во векторска репрезентација за тренирање на граф-невронски мрежи кои се состојат од граф-конволуциски слоеви, DMoNPooling и K-димензионални граф-невронски слоеви за финална екстракција на карактеристики и класификација на читливост на код. Добиваат 72.5% и 88% во тро- и дво-класен формат на проблемот.

Околу проблемот за детекција на AI-генериран код има неколку пристапи но нема дефинитивен. [10] врши емпириска оценка на различни постоечки детектори за AI-генерирана содржина: GPTZero, Sapling, GPT-2 Detector, DetectGPT, GLTR [11, 12, 13, 14, 15, цитирани во [10]]. Нивниот заклучок е дека додека е возможна детекција на AI-генериран код со вакви алатки, тоа многу зависи од типот на код кој се класифицира и од самиот модел кој ја врши класификацијата, со резултати кои се движат од 48.41% прецизност до 76.93% прецизност кај некои модели. Подоцна се појавува [16], каде авторите предлагаат метод во кој користејќи контрастивно учење вршат fino подесување на UniXcoder модел [17], а потоа вршат класификација на векторите кои тој ги произведува користејќи целосно поврзана мрежа—класификатор—врз податочно множество кое се состои од 228 илјади парови Python код и 222 илјади парови Java код. Добиваат одлични резултати врз Python примероците со 99.2% прецизност, и малку послаби врз Java примероците со 96.7% прецизност.

Паралелно на методите за длабоко учење, популарни се и традиционалните модели на машинско учење. Во [18] освен што креираат податочно множество за токму оваа употреба, вршат анализа на разни модели од машинското учење (случајни шуми, XGB, SVM и баесов класификатор), добивајќи 71% прецизност со најдобриот модел. Паралелно, во [19] креираат податочно множество и го тестираат освен на модели од машинското учење со кои добиваат највисок резултат од 89%, дополнително тестираат и класификација на вектори добиени од модели како CodeBERT [20], UniXcoder и CodeT5+ [21], најдобрите резултати добиени врз векторите од UniXcoder со 98.63%.

Горенаведените методи ја демонстрираат ефективната на методи базирани на

синтаксни дрва за анализа и класификација на код, но таргетираат потрадиционални проблеми во сферата на код класификација, имено детекција на клонови, одредување читливост, сумаризација. Методите претставуваат јасен заклучок дека користењето на само апстрактни синтаксни дрва е често недоволно за решавање на наведените проблеми, туку е потребно нивно збогатување. Дополнително, нема многу истражување во користењето на вакви дрва во детекцијата на AI-генериран програмски код, што претставува очигледна дупка во науката која се фокусира на класификација на код. Овој проблем е најчесто решаван користејќи токен-базирани пристапи како класификација на ембединг-вектори добиени од некој претрениран или фино-подесуван модел, или пак користејќи статични карактеристики за тренирање на традиционални модели од машинското учење.

Во овој труд вршиме евалуација на повеќе различни методи, 2 од кои претставуваат пристапи кои не се досега испробани конкретно врз проблемот за идентификација на AI-генериран програмски код. Едниот пристап е заснован на користејќи конволуциски мрежи заедно со двонасочна LSTM мрежа, додека другиот прави класификација на конкретни синтаксни дрва.

3 Методологија

Во овој дел ги претставуваме различните методи за класификација кои ќе ги споредуваме, како и податоците врз кои работиме. Ќе започнеме прво со податоците, нивната припрема и детали, а во продолжеток следат различните методи заедно со нивните спецификации.

3.1 Експериментална конфигурација

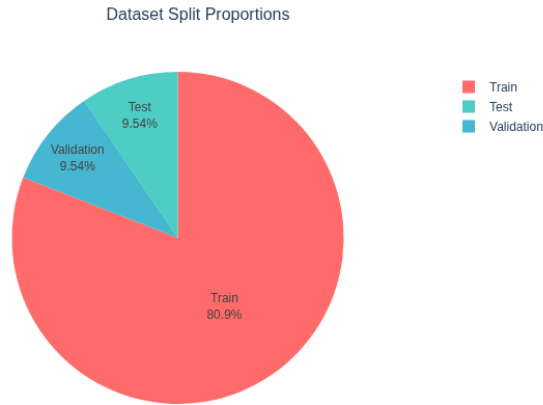
За да овозможиме фер споредба и репродукција на експериментите, сите експерименти користат фиксиран random seed: 872002. Експериментите ги користат истите податоци за тренирање, валидација и тестирање кои се опишани во дел 3.2.

3.2 Податоци и нивна припрема

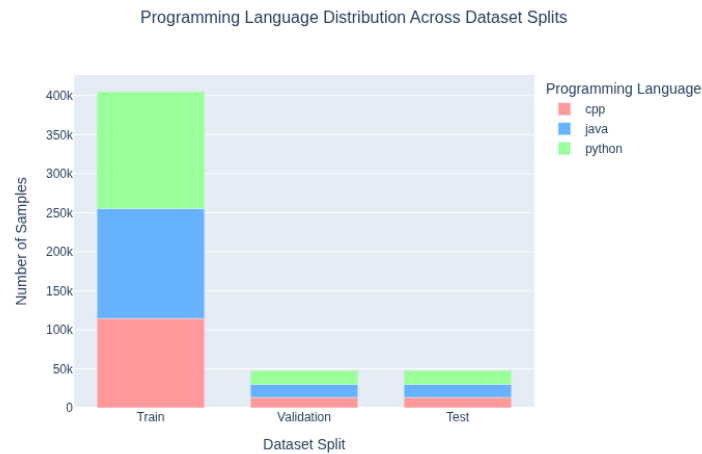
Со цел фер споредба на методите, во рамките на целиот труд се користи едно исто податочно множество. CoDeT-M4 [19] е податочно множество кое се состои од 500,552 примероци код од кои 246,221 припаѓаат на класата *human* и 254,331 се распоредени помеѓу 5-те класи за AI модели. Кодовите се во 3 програмски јазици: Java, Python и C++. За наша примена тие 5 класи се агрегиарни во една класа *ai*. Овие примероци се распоредени во три подмножества: *train*, *test* и *validation*. Слика 1 го прикажува односот на примероци во секое подмножество, додека во слика 2 е прикажана застапеноста на секој програмски јазик во рамките на секое подмножество. Сликата 3 прикажува распределба на должините на кодовите во множеството, во која се покажува дека

Податочното множество исто така нуди и неколку карактеристики за секој примерок код, како што се *avgFunctionLength*, *avgIdentifierLength*, *avgLineLength* и други, но поради целите на ова истражување, тие не беа искористени.

Во текот на истражувањето и анализа на податочното множество беше приметено преклопување на дел од примероците кои се појавуваат во *train* подмножеството, се појавуваа и во едно од другите две. Поточно, постоеја 1234 поединечни инстанци



Слика 1: Пропорција на секое подмножество



Слика 2: Распореденост на јазици во секое подмножество

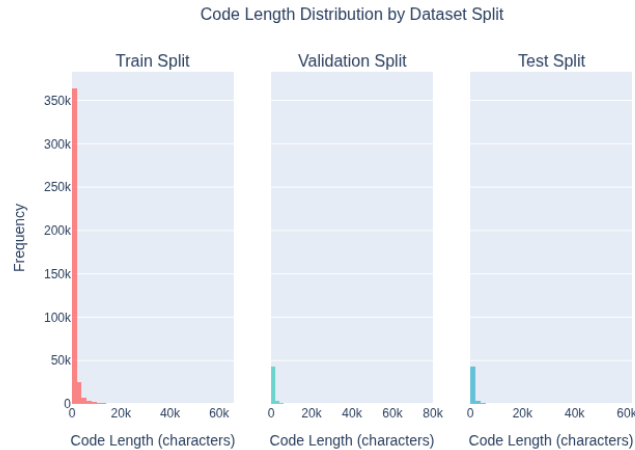
во кои се појавуваа примероци во повеќе од едно подмножество. Имаше и ситуации кај кои во сите три делови од множеството се појавуваше еден ист примерок. Пред вршење на експериментите извршено е чистење на таквите примероци, со тоа што се отстранети од *val* и *test* деловите. Слика 4 го прикажува степенот на преклопување, заедно со информација за засегнатите делови од податочното множество.

3.2.1 Креирање карактеристики

Овој дел објаснува какви карактеристики се користат за методите опишани во делот 3.3.

3.2.1.a Креирање синтаксни дрва

Синтаксните дрва се една од најраните техники за репрезентација и анализа на код. Во ова истражување централна алатка за креирање на конкретни синтаксни дрва (исто така познати како парсирачки дрва) е *Tree-Sitter* [22]. Алатката овоз-



Слика 3: Дистрибуција на должини на кодовите во секое подмножество

можува парсирање на мноштво различни програмски со иницијализација соодветната граматика на таргет-јазикот. По креирање на парсер инстанцата можеме да испраќаме барања (анг: queries) за да ги добиеме информациите кои ни се потребни, или пак да го преземеме целосното конкретно синтаксно дрво за процесирање понатаму.

3.2.1.6 Репрезентација на податоците за традиционалните методи од МУ

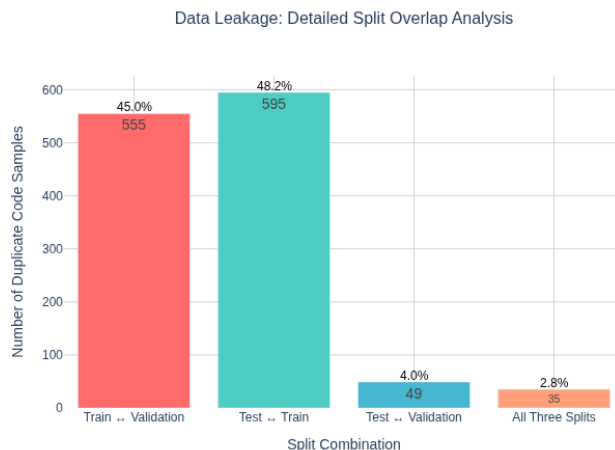
За методите од машинското учење кои ги опишуваме во дел 3.3.1 потребно е креирање на карактеристики кои ќе бидат репрезентација на секој примерок код. За таа цел, по креацијата на синтаксното дрво, која е опишана во дел 3.2.1.a, праќаме барања со кои за секој примерок код ги добиваме карактеристиките опишани во табела 1. По креација на карактеристиките тие се нормализирани користејќи *StandardScaler* од библиотеката *Scikit-Learn* [23].

Карактеристика	Објаснување
Functions	Број на функциски или метод декларации
Classes	Број на класа и интерфејс декларации низ кодот
If Statements	Број на if-изрази
Loops	Број на циклуси (while, for, do-while)
Imports	Број на импорт декларации
Comments	Број на коментари (еднолиниски или повеќелиниски)
Binary Operations	Број на бинарни изрази (аритметички, логички или споредбени оператори)
Errors	Детектира дали кодот имал синтаксна грешка во текот на парсирањето

Табела 1: Карактеристики екстрахирани од секое синтаксно дрво и нивни објаснувања

3.2.1.в Креирање графови од синтаксни дрва

Претворање на синтаксните дрва во графови кои можат да бидат обработени од граф-невронските мрежи е клучен чекор во нашето истражување. По креирањето на едно конкретно синтаксно дрво (опишано во дел 3.2.1.a, потребно е да се претвори во формат соодветен за граф неврронските мрежи. Согласно имплементацијата



Слика 4: Број на примероци кои се појавуваат во соодветната комбинација од подмножествата. Процентот над секој графикон означува колкав процент од вкупниот број дупликат инстанци се јавуваат во соодветната комбинација подмножества.

на моделите опишана во дел 3.3.2, графовите ги креираме со библиотеката Pytorch-Geometric [24, 25]. Прво, се изминува целото дрво и се екстрахираат типовите од секој јазел во дрвото. Тие типови се мапираат од текстуална форма (пример, *if_statement*) во целоброен *ID*. Овие броеви потоа претставуваат атрибути на секој јазел, додека на ребрата не поставуваме никаков атрибут. Со ваквата репрезентација на јазлите моделите добиваат семантичко значење од дрвото (графот), додека преку дрво-структурата, односно ребрата, моделите добиваат структурно значење на секој јазел, односно дел од кодот. Врските во графот се насочени за да се задржи директната parent-child структура на кодот, на пример при *if* израз јазлите кои припаѓаат на телото на изразот во кодот се директно зависни од самиот израз.

3.2.1.в.1 Видови граф-базирани множества

Во рамките на ова истражување креираме неколку различни верзии од граф-множеството. Табела 2 ги опишува различните видови.

Тип на множество	Објаснување
Base	Основна верзија на граф-базираното множество како што е опишано во 3.2.1.в, нема никакви претпроцесирања
Comments	Граф-базирано множество со задржани јазлите кои означуваат коментари низ кодот, но не и содржината (текстот) од самиот коментар
Cleaned	Основното множество со отстранети дупликати во val и test подмножествата
Comments + Cleaned	Задржани коментар-јазли и отстранети дупликати
Cleaned + Comments + Depth & Positional Embeddings	Задржани коментар-јазли, исчистено од дупликати, јазлите се дополнети со нови атрибути за длабочина во дрвото и child-index од јазелот

Табела 2: Сите видови граф множества кои беа креирани за користење во ова истражување заедно со објаснувања за истите

3.3 Методи

3.3.1 Традиционално машинско учење

Овој поддел ги прикажува моделите од машинско учење кои ги евалуираме во овој труд. Податоците врз кои се тренираат и тестираат се опишани во дел 3.2.1.6. Имплементацијата на сите модели е искористена од Scikit-Learn библиотеката. Во табела 3 се опишани моделите кои се користени заедно со нивните хиперпараметри. Останатите хиперпараметри кои не се споменати во табелата се поставени на почетната вредност како што се зададени во библиотеката.

Модел	Хиперпараметри	Вредности
Random Forest	n_estimators: [10, 20, 30, 50] max_depth: [10, 20, 30, 40] min_samples_split: [2, 3, 5]	30 20 5
Logistic Regression	C: [0.01, 0.1, 1, 10] penalty: [l1, l2] solver: [liblinear, saga]	0.01 l2 liblinear
Naive Bayes (Gaussian)	var_smoothing: [$1e^{-9}$, $1e^{-8}$, $1e^{-7}$, $1e^{-6}$]	$1e^{-9}$
CatBoost	iterations: [100, 200, 300] learning_rate: [0.03, 0.1, 0.2] depth: [4, 6, 8] l2_leaf_reg: [1, 3, 5]	200 0.1 4 3

Табела 3: Традиционални модели од машинско учење и нивните хиперпараметри.

3.3.2 Длабоко учење

Овој поддел ги презентира и објаснува методите кои се засноваат на модерни техники на длабоко учење. Ќе ги објасниме различните методи почнувајќи со стандардни токен-базирани методи на класификација, а потоа и методите кои се засноваат на класификација на апстарктни синтаксни дрва користејќи граф-невронски мрежи. Сите модели кои се дискутирани во овој дел се изградени со PyTorch [26] и PyTorch-Geometric [24, 25]. Оптимизатор за сите модели е Адам оптимизаторот [27].

3.3.2.a Токен базирани методи

Во овој поддел се прикажани методите кои вршат класификација на векторски репрезентации на токени од кодот.

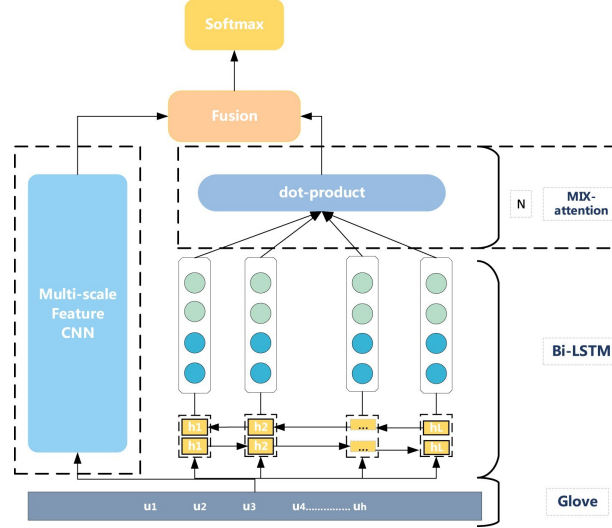
3.3.2.a.1 CodeBERT Embeddings classification

Овој метод претставува основна класификација на вектори кои се добиени од CodeBERT [20] модел. Креиран е модел кој во себе го има CodeBERT моделот како еден слој, од кој се добиваат 768-димензионални вектори. Тие вектори потоа се класифицирани од целосно поврзана мрежа со еден слој.

3.3.2.a.2 Multi-Scale CNN + Bi-LSTM

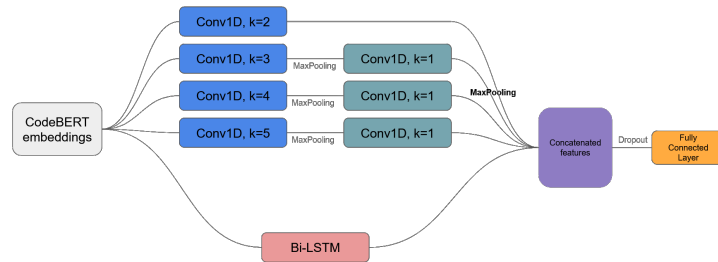
CBM моделот, предложен во [4], комбинира мрежа со конволуции со повеќе скалирања (MCNN) и дводнасочна LSTM мрежа (Bi-LSTM) за подобрување на класификацијата на текст преку екстракција на локални и глобални семантички карактеристики. MCNN модулот користи конволуциски кернели со различни големини

(2×300 , 3×300 , 4×300 , 5×300) за да извлече локални карактеристики од текст векторизиран со *GloVe* [28], додека Bi-LSTM модулот ги доловува глобалните зависности во текстот. Овие карактеристики се спојуваат и се проследуваат низ *softmax* слој за класификација. Конфигурацијата на оригиналната мрежа е прикажана на слика 5.



Слика 5: Multi-Scale CNN + Bi-LSTM мрежа. Извор: [4].

Во овој труд, мрежата е адаптирана за обработка на код со тоа што направена е замена на *GloVe* со 768-димензионални векторски репрезентации од претрениран *CodeBERT* модел. Замената е направена поради тоа што *CodeBERT* е специфично трениран на програмски јазици. Векторите паралелно се обработуваат во MCNN мрежата и Bi-LSTM слојот. MCNN мрежата користи паралелни конволуциски слоеви со кернели со големина од 2 до 5 за да извлече локални карактеристики кои би биле слични на би-, три-, четири- и пет-грами. Извлечените карактеристики дополнително се обработуваат од секундарни паралелни слоеви. По обработка од соодветните слоеви, извлечените карактеристики се спојуваат, и потоа поминуваат низ *dropout* слој пред да бидат обработени од целосно поврзана мрежа за класификација. Адаптираната архитектура е прикажана на слика 6.



Слика 6: Multi-Scale CNN + Bi-LSTM мрежа адаптирана за нашиот проблем

3.3.2.a.3 Само Multi-Scale CNN

Со цел споредба дали Bi-LSTM слојот контрибуира кон подобра класификација на примероците во множеството, извршивме и тренирање само на MCNN делот од мрежата опишана во дел 3.3.2.a.2. Конфигурацијата на тие слоеви е иста како и во целосниот модел, со тоа што класификаторот во овој модел има помала бројка на влезни параметри поради отстранувањето на Bi-LSTM карактеристиките од конечната латентна репрезентација на секој примерок во множеството.

3.3.2.б Граф-базирани методи

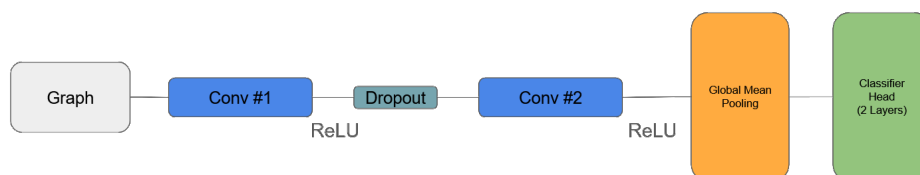
Овој поддел ги презентира методите кои се засноваат на градење графови од конкретните синтаксни дрва опишани во дел 3.2.1.a. Клучно во овие граф-базирани методи е тоа што во секој модел постои *Embedding* слој со кој се мапираат типовите на секој јазел во векторски репрезентации кои моделот ги учи во текот на тренирањето. Со тоа се постигнува учење не само на самата структура на графовите туку и на содржината која тие ја репрезентираат.

3.3.2.б.1 Граф-конволуциска мрежа

Граф-конволуциските мрежи се невронски мрежи чија задача е обработка на структурирани граф-базирани податоци. Ваквите мрежи нудат способност за моделирање на врските помеѓу ентитети - јазли во графот поврзани меѓусебно со ребра. За разлика од традиционални невронски мрежи, граф-конволуциските мрежи користат агрегација на информациите кои се зачувани во соседните јазли на даден јазел со информациите од тој јазел за да ја креираат репрезентацијата на јазелот.

Во овој експеримент извршувавме и оптимизација на хиперпараметрите, и како хиперпараметар го третиравме и типот на конволуциски слоеви, имено, освен GCN слоевите опишани во [29], како опција за оптимизација ги вклучивме и SAGE конволуциските слоеви опишани во [30]. Клучната разлика помеѓу двата типа конволуциски слоеви е тоа што GCN слоевите ја земаат целосната матрица на соседност на графот за да направат агрегации на соседите од даден јазел, додека SAGE слоевите користат фиксен број на соседи, што ги прави подобри за обработка на големи графови или графови од различни големини, какви што се графовите во ова истражување. Целосната листа на оптимизирани хиперпараметри заедно со оптимизираните вредности се наоѓа во табела 4.

Моделот функционира така што на влез ги добива карактеристиките од секој јазол, во нашиот случај тоа би биле целобројните *IDs* кои го претставуваат неговиот вид, и ребрата со кои јазлите се поврзани. Пред тие да бидат пропагирани низ граф-конволуциските слоеви, карактеристиките се пропагираат низ ембединг слојот со што се добиваат векторски репрезентации за типот на секој јазел. Потоа овие вектори, заедно со индексот на ребрата, пропагираат низ конволуциските слоеви, меѓу кои стои слој за dropout за превенција на overfitting. По добивање на латентните карактеристики од вториот конволуциски слој, врз нив се врши global mean pooling за да се добие единствена векторска репрезентација за целиот граф. Конечно, добиените карактеристики по pooling чекорот преминуваат низ целосно поврзана мрежа од која се добиваат веројатностите за класата на која ѝ припаѓа графот, односно примерокот код. Архитектурата на мрежата е прикажана во слика 7. f



Слика 7: Граф-конволуциска мрежа

Хиперпараметар	Ранг на вредности	Оптимизирана вредност	Објаснување
Hidden Dim 1	[128, 512]	512	Димензија на првиот граф-конволуциски слој
Hidden Dim 2	[128, 512]	400	Димензија на вториот граф-конволуциски слој
Embedding Dim	[64, 512]	176	Димензија на ембединг векторите за јазлите
Use SAGE	True, False	True	Избор меѓу GCN [29] или SAGE [30] слоеви
Learning Rate	[0.0001, 0.01]	0.0012697	Рата на учење за Adam оптимизаторот
Classifier Hidden Dim 1	Половина од Hidden Dim 2	N/A	Димензија на првиот скриен слој во класификаторот
Classifier Hidden Dim 2	Половина од Classifier Hidden Dim 1	N/A	Димензија на вториот скриен слој во класификаторот

Табела 4: Хиперпараметри за граф-конволуцискиот модел

Хиперпараметрите наведени во табела 4 се добиени користејќи global mean pooling за агрегација на карактеристиките од графовите. По добивање на вредности за овие хиперпараметри направени беа дополнителни тестирања користејќи Attention Pooling [31], кој користи невронска мрежа за да одреди на кои карактеристики да им означи најголеми тежини, и global max pooling.

3.3.2.6.2 Graph-attention Мрежа

Овој метод се заснова на замена на конволуциските слоеви споменати во дел 3.3.2.6.1 со Graph Attention (GAT) слоеви. Првично предложени во [32], за разлика од GCN слоевите, кои ја креираат репрезентацијата на јазел врз основа на спектралните својства на графот, или GraphSAGE, кој агрегира карактеристики од фиксен број соседи со еднаква тежина, GAT слоевите користат механизам на attention за споредување на карактеристиките на даден јазел со неговите соседи. Ова овозможува доделување различни тежини на соседите при пресметка на новата репрезентација на јазелот. Останатиот дел од архитектурата останува ист како и во слика 7.

3.3.2.6.3 Graph Transformer Мрежа

Во овој метод користиме Graph Transformer слоеви, предложени во [33]. За разлика од GCN, кои агрегираат карактеристики врз основа на спектрални својства,

GraphSAGE, кој комбинира фиксен број соседи давајќи им еднаква тежина, или GAT, кои користат attention само за карактеристики, Graph Transformer применува multi-head attention за комбинирање на карактеристиките и лабелите на јазлите, вклучувајќи ги и карактеристиките на врските. Ова овозможува пофлексибилно моделирање на релациите меѓу јазлите.

Конфигурацијата на архитектурата е иста како во слика 7, со тоа што се заменети граф-конволуциските слоеви со граф-трансформер слоеви. Користиме 8 attention heads во секој трансформер слој. Користење карактеристики на врски е исклучено поради тоа што во нашите графови немаме никаква карактеристика која стои на врската бидејќи врските ни служат само за репрезентација на структурата на кодот, како што е опишано во 3.2.1.a.

За овој метод беа испробани две верзии на мрежата:

1. **Граф-трансформер мрежа без позиционални ембединзи на јазлите** За оваа цел беше искористено стандардното множество (исчистено) + јазли за коментари опишано во табела 2.
2. **Граф-трансформер мрежа со вклучени позиционални ембединзи на јазлите** За оваа цел беше искористено множеството со вклучени позиционални ембединзи за длабочина и реден број на дете-јазел опишано во табела 2.

4 Резултати

Во овој дел ги презентираме резултатите од методите опишани во дел 3.3 кои се евалуирани врз CoDeT-M4 податочното множество, опишано во дел 3.2. Резултатите се поделени на три дела: перформанси на традиционалните методи од машинско учење, перформанси на токен-базираните методи на длабоко учење, и перформанси на граф-базираните методи кои користат конкретни синтаксни дрва (CST). Сите експерименти се изведени со фиксиран *random seed* (872002) за репродуктивност, како што е наведено во 3.2.

Во ова истражување користиме повеќе метрики со цел да обезбедиме целосна слика за перформансите на различните модели. Точноста (*Accuracy*) ја мери вкупната коректност на класификацијата, додека чувствителноста (*Recall*) ја оценува способноста на моделот да ја препознае позитивната класа *ai*. F1-мерката ги комбинира прецизноста и чувствителноста со хармонична средина, со што дава баланс помеѓу двете. Прецизноста (*Precision*) го оценува процентот на вистински позитивни предвидувања од сите позитивно предвидени примери, а специфичноста (*Specificity*) ја мери способноста на моделот да ја разликува негативната класа *human*. Дополнително, кривата под ROC (*AUROC*) претставува глобална мерка на дискриминативната моќ на моделот низ сите можни прагови на одлука.

4.1 Перформанси на традиционалните методи од машинско учење

Целта на експериментот со методите опишани во дел 3.3.1 е да се воспостави некаква почетна линија за споредба, но истовремено и да се види дали избраните статични карактеристики извлечени од синтаксни дрва (опишани во табела 1 се доволно експресивни за идентификација на AI-генериран програмски код.

Согласно резултатите опишани во табела 5, најдобрите модели се случајна шума

Модел	Accuracy	Recall	F1	Precision	Specificity	AUROC
Random Forest	0.7374	0.6930	0.7210	<u>0.7513</u>	0.7800	0.8242
Logistic Regression	0.6329	0.6422	0.6314	0.6209	0.6240	0.6819
Naive Bayes	0.6083	0.5151	0.5628	0.6203	0.6976	0.6561
CatBoost	<u>0.7363</u>	<u>0.6597</u>	<u>0.7101</u>	0.7688	<u>0.8097</u>	<u>0.8223</u>

Табела 5: Перформанси на традиционалните ML модели на *test* подмножеството од CoDeT-M4. **Болдирано** = најдобро, подвлечено = второ најдобро.

и CatBoost. Блискоста во резултатите на овие 2 модели се должи на тоа што и двата модели се добри за учење нелинеарни зависности. Случајните шуми со големата бројка на дрва, а и CatBoost со тоа што тој е форма на Gradient Boosting која во позадина користи одлучувачки дрва, кои исто така се добри за нелинеарни зависности. Наивниот баесов класификатор, пак, има најлоши резултати токму поради неговата наивност: карактеристиките *не* се независни, па не може добро да ги моделира. Логистичката регресија се наоѓа измеѓу двата модели според добиените перформанси поради тоа што постојат и некакви линеарни зависности во карактеристиките, меѓутоа тие не се доволни за да се постигнат повисоки резултати.

Резултатите од овој метод се незадоволителни. Дури и најдобрите модел не можат добро да ги распознаваат класите и се лоши и во точно класифицирање на негативната класа *human*, како и во точна класификација на критичната, позитивната класа, *ai*. Ова означува дека карактеристиките добиени од конкретни синтаксни дрва се недоволно експресивни и недоволно ја претставуваат разликата помеѓу класите.

4.2 Перформанси на токен-базираните методи за длабоко учење

Овој дел ги презентира резултатите од евалуацијата методите опишани во дел 3.3.2.а. Целта на овие методи е да се разгледа дали вектори добиени од модел претерниран на код (во нашиот случај CodeBERT) се доволно експресивни за класите да бидат јасно одделени во векторскиот простор, а воедно и дали истите вектори се доволно генерални за да може да се направи класификатор кој е агностичен кон различни LLMs и програмски јазик.

Model	Accuracy	Recall	F1	Precision	Specificity	AUROC
Baseline (only CodeBERT embeddings)	0.8452	0.8393	0.8404	0.8414	0.8507	0.9193
Baseline with 2-layer classifier	0.9436	0.9289	0.9416	0.9547	0.9577	0.9847
Multi-Scale CNN + Bi-LSTM on CodeBERT embeddings	0.9836	0.9837	0.9836	0.9835	0.9837	0.9837
Only Multi-Scale CNN	<u>0.9734</u>	<u>0.9735</u>	<u>0.9734</u>	<u>0.9733</u>	<u>0.9735</u>	<u>0.9735</u>

Табела 6: Перформанси на токен-базираните методи на *test* подмножеството. **Болдирано** = најдобро, подвлечено = второ најдобро.

Векторските репрезентации од CodeBERT [20] се покажаа како ефективна репрезентација на податочното множество CoDeT-M4, овозможувајќи добро раздвојување на класите *ai* и *human* во сите токен-базирани модели (дел 3.3.2). MCNN мрежата постигнува малку пониски резултати во споредба со MCNN дополнета со Bi-LSTM, што укажува дека конволуциските слоеви со различни големини на кернели (2 до 5) успешно извлекуваат локални карактеристики од CodeBERT ембединзите, слично на n-грами во обработката на природни јазици, но го губат глобалниот контекст. Овие

слоеви ефикасно фаќаат локални зависности како поврзаноста помеѓу типот, името и вредноста на варијабла во C++. Додавањето на Bi-LSTM овозможува моделирање на подолги зависности, како декларација и инстанцирање на класа на различни места во кодот, што резултира со подобрување од 1% на сите метрики (Табела 6). Спротивно на ова, директната класификација на CodeBERT ембединзите со еден целосно поврзан слој дава послаби перформанси поради ограничената способност за извлекување експресивни карактеристики, додека класификатор со два слоја значително подобро ги учи разликите меѓу класите.

4.3 Перформанси на граф-базираните методи

Овој дел ги презентира резултатите од евалуацијата на граф-базираните методи опишани во дел 3.3.2.б, кои користат конкретни синтаксни дрва (CST). Целта на овие методи е да се испита способноста на граф-невронските мрежи да извлекуваат структурни и семантички карактеристики од CST графови (дел 3.2.1.в, Табела 2), со што ќе се овозможи прецизна класификација на AI-генериран наспроти човечки напишан код. Исто така, се разгледува дали овие методи можат да бидат агностични кон различни програмски јазици и LLMs само преку карактеристиките научени од конкретните синтаксни дрва.

Model (Податоци/конфигурација)	Accuracy	Recall	F1	Precision	Specificity	AUROC
GCN (Cleaned/Mean Pooling)	0.8841	0.8620	0.8801	0.8971	0.9052	0.9509
GCN + Comments Cleaned + Mean Pooling	0.9453	0.9280	0.9432	0.9589	0.9619	0.9849
GCN (Comments/Max Pooling)	0.9300	0.9096	0.9272	0.9454	0.9497	0.9797
GCN (Comments/Attention Pooling)	<u>0.9377</u>	0.9138	<u>0.9349</u>	<u>0.9571</u>	<u>0.9607</u>	<u>0.9821</u>
GAT (Comments/Mean Pooling)	0.8794	0.8379	0.8718	0.9086	0.9192	0.9501
Graph Transformer (Cleaned/No positional embeddings)	0.9201	0.9034	0.9165	0.9301	0.9359	0.9744
Graph Transformer (Cleaned + Comments/No positional embeddings)	0.9348	0.9122	0.9319	0.9525	0.9564	0.9807
Graph Transformer (Cleaned + Comments + Positional Embeddings)	0.9351	<u>0.9156</u>	0.9325	0.9499	0.9537	0.9812

Табела 7: Перформанси на граф-базираните методи на *test* подмножеството **Болдирано** = најдобро, подвлечено = второ најдобро. Колоната за модел во заграда означува кои податоци се користат, опишани во табела 2, и која конфигурација на мрежата се користи, опишани во дел 3.3.2.б.

Согласно резултатите презентирани во табела 7, се покажува дека моделите имаат голем бенефит од јазлите кои означуваат коментари. (Напомена: Јазлите не содржат во себе репрезентација на содржината од коментарот, само означуваат дека тука стои коментар во оригиналниот код.) Повеќето модели покажуваат одлични перформанси за проблемот. Најдобрите модели меѓусебно се разлукваат за само мал процент во секоја метрика, со тоа што граф-конволуциската мрежа опишана во дел 3.3.2.б.1 има највисоки резултати на секоја метрика, освен кај прецизност и специфичност, каде губи за помалку од половина процент од истата мрежа но со Attention Pooling како метод на агрегација. Разликата помеѓу mean pooling и attention pooling варијантите

се должи на тоа што веројатно сите својства на графовите се важни, а attention механизмот означува помала важност на некои својства со што се губи нивната информација. Од истата причина GAT мрежата ги има најслабите резултати; механизмите кои ги користи не одредуваат соодветна важност на сите карактеристики, со што се губат важните информации.

Граф-трансформер мрежите уживаат бенефит од позиционалните и длабочинските ембединзи, но сепак не успеваат да ја надминат граф-конволуциската мрежа. Можно објаснување е тоа што трансформерите имаат глобален self-attention механизам кој треба да научи на кои делови од графовите треба да се фокусира, што слично како и кај Attention Pooling и GAT, не успева целосно да научи кои/какви јазли носат најмногу информација.

4.4 Споредбена анализа

Модел	Accuracy	Recall	F1	Precision	Specificity	AUROC
Традиционални ML модели						
Random Forest	0.7374	0.6930	0.7210	0.7513	0.7800	0.8242
Logistic Regression	0.6329	0.6422	0.6314	0.6209	0.6240	0.6819
Naive Bayes	0.6083	0.5151	0.5628	0.6203	0.6976	0.6561
CatBoost	0.7363	0.6597	0.7101	0.7688	0.8097	0.8223
Токен-базирани методи						
Baseline (only CodeBERT embeddings)	0.8452	0.8393	0.8404	0.8414	0.8507	0.9193
Baseline with 2-layer classifier	0.9436	0.9289	0.9416	0.9547	0.9577	0.9847
Multi-Scale CNN + Bi-LSTM on CodeBERT embeddings	0.9836	0.9837	0.9836	0.9835	0.9837	0.9837
Only Multi-Scale CNN	0.9734	0.9735	0.9734	0.9733	0.9735	0.9735
Граф-базирани методи						
GCN (Cleaned/Mean Pooling)	0.8841	0.8620	0.8801	0.8971	0.9052	0.9509
GCN + Comments Cleaned + Mean Pooling	0.9453	0.9280	0.9432	0.9589	0.9619	0.9849
GCN (Comments + Cleaned/Max Pooling)	0.9300	0.9096	0.9272	0.9454	0.9497	0.9797
GCN (Comments + Cleaned/Attention Pooling)	0.9377	0.9138	0.9349	0.9571	0.9607	0.9821
GAT (Comments + Cleaned/Mean Pooling)	0.8794	0.8379	0.8718	0.9086	0.9192	0.9501
Graph Transformer (Cleaned/No positional embeddings)	0.9201	0.9034	0.9165	0.9301	0.9359	0.9744
Graph Transformer (Cleaned + Comments/No positional embeddings)	0.9348	0.9122	0.9319	0.9525	0.9564	0.9807
Graph Transformer (Cleaned + Comments + Positional Embeddings)	0.9351	0.9156	0.9325	0.9499	0.9537	0.9812

Табела 8: Перформанси на сите модели на *test* подмножеството од CoDeT-M4. **Болдирано** = најдобар резултат низ сите модели.

Според резултатите во табела 8, статичните карактеристики изведени од синтаксните дрва се несоодветна репрезентација на податочното множество, што укажува на потреба да се изнајдат поинакви статични карактеристики, како на пример оние опишани во [19], каде се добива 89.04% прецизност со CatBoost, за разлика од 73.63%

со CatBoost во нашето истражување. Додека таа прецизност сепак не е доволно висока за коректна идентификација на AI-генериран код, сепак е повисока од нашиот најдобар традиционален модел од машинското учење, случајната шума, која добива само 73.74% прецизност.

Од друга страна, резултатите од токен-базираните методи се премногу задоволителни. Освен што од нив се покажува дека подлабоките невронски мрежи се посоодветни за учење на карактеристиките во ембединг векторите (моделот Baseline with 2-layer classifier), се покажа дека Multi-Scale конволуциските мрежи се одлични во екстрахирање експресивни карактеристики од овие ембединг вектори, а моделирањето на долгите зависности користејќи двонасочна LSTM мрежа доведува до уште подобро класифицирање, што укажува на тоа дека локалните и глобалните зависности во код се подеднакво важни при детекција на AI-генериран код.

Потоа, граф-невронските мрежи се уникатен и ефективен пристап кон решавање на овој проблем, како што покажуваат резултатите. Додека не се покажаа како најдобар метод во ова истражување, сепак имаат високи резултати, што укажува на тоа дека мрежите можат да научат значителни карактеристики од конкретни синтаксни дрва.

5 Идни истражувања

Нашиот труд се залагаше да претстави споредба на различни методи за детекција на AI-генериран код. Во рамките на овој труд, освен деталната споредба на различни модели од традиционалното машинско учење, како и методи кои се засноваат на методи од длабоко учење, претставен е и уникатен пристап за класификација на програмски код базиран на конкретни синтаксни дрва (CST). За таа цел, предлагаме неколку идни насоки за доистражување на овој проблем:

- **Збогатување на конкретните синтаксни дрва** Како што покажуваат истражувањата кои се засноваат на користење *апстрактни* синтаксни дрва [8, 34, 7], збогатувањето на истите со различни видови врски дава значителни подобрувања во перформансите. Од таа гледна точка предлагаме збогатување и на конкретните синтаксни дрва со слични елементи.
- **Кастрење на синтаксните дрва** Синтаксните дрва стануваат подлабоки и пошироки со секоја дополнителна линија код. Тоа може да доведе до губење на значењето на некои структури и конструкции во кодот, со што се намалуваат перформансите на граф-конволуциските мрежи. Наш предлог е да се направи истражување во кое би се кастреле синтаксните дрва од јазли кои не носат голема информација (јазли во кои има пунктуација, гранење, услови во контролните структури, и сл. Со ова би се намалила значително димензионалноста на синтаксните дрва (конкретни или апстрактни) и ќе се отстрани шум од податоците.
- **Користење ембединг вектори од јазичен модел како атрибути на јазлите** Атрибутите на јазлите во ова истражување беа само видот на структура од код која тие ја репрезентираат (услов, циклус, бинарна операција,...) Иако овие податоци се покажаа дека имаат силна експресивност, сепак не ги достигнаа способностите на длабоките невронски мрежи. За таа цел, збогатување на јазлите со ембединзи од јазичен модел (пр. CodeBERT) е добра насока за понатамошни истражувања.

6 Заклучок

Појавата на AI-генериран код претставува проблем кој има разни негативни импликации: етички, едукативни, дури и легални. Ова истражување споредува различни методи за класификација на AI-генериран наспроти човечки напишан код, при што воведува нов пристап базиран на конкретни синтаксни дрва (CST) генерирани со Tree-sitter [22], претворени во насочени графови и класифицирани со граф-невронски мрежи (GCN, GAT, Graph Transformer) [29, 32, 33]. Резултатите покажуваат дека карактеристики базирани на овие синтаксни дрва се несоодветна репрезентација на класите, што се рефлектира и во перформансите на случајните шуми, наивниот баесов класификатор, CatBoost моделот и логистичката регресија. Додека токен-базирани методи се потврдуваат како најдобри, согласно со истражувањата во [16], граф-базирани методи не се далеку зад нив. Пристапот базиран на CST, за кој не најдовме претходни истражувања кои ги користат таквите дрва, се покажува како ефективен и ефикасен, и ја нагласува важноста на структурните разлики и зависности на програмскиот код. Од резултатите исто така се гледа дека е потребно дополнително истражување во добри, издржани и широко-репрезентативни множества за AI-генериран наспроти човечки напишан код, бидејќи и покрај одличните резултати постигнати во ова истражување, сепак проблемите кои ги предизвикува сè поголемото користење на големите јазични модели за програмирање немаат цврсто решение и одговор.

7 Библиографија

- [1] Juyong Jiang, Fan Wang, Jiasi Shen, Sungju Kim, and Sunghun Kim. A survey on large language models for code generation, 2024. URL: <https://arxiv.org/abs/2406.00515>, arXiv:2406.00515.
- [2] Brett A. Becker, Paul Denny, James Finnie-Ansley, Andrew Luxton-Reilly, James Prather, and Eddie Antonio Santos. Programming is hard - or at least it used to be: Educational opportunities and challenges of ai code generation. In *Proceedings of the 54th ACM Technical Symposium on Computer Science Education V. 1*, SIGCSE 2023, page 500–506, New York, NY, USA, March 2023. Association for Computing Machinery. URL: <https://dl.acm.org/doi/10.1145/3545945.3569759>, doi:10.1145/3545945.3569759.
- [3] Franco Scarselli, Marco Gori, Ah Chung Tsoi, Markus Hagenbuchner, and Gabriele Monfardini. The graph neural network model. *IEEE Transactions on Neural Networks*, 20(1):61–80, January 2009. doi:10.1109/TNN.2008.2005605.
- [4] Hai Huan, Zelin Guo, Tingting Cai, and Zichen He. A text classification method based on a convolutional and bidirectional long short-term memory model. *Connection Science*, 34(1):2108–2124, December 2022. doi:10.1080/09540091.2022.2098926.
- [5] Weisong Sun, Chunrong Fang, Yun Miao, Yudu You, Mengzhe Yuan, Yuchen Chen, Quanjun Zhang, An Guo, Xiang Chen, Yang Liu, and Zhenyu Chen. Abstract syntax tree for programming language understanding and representation: How far are we? (arXiv:2312.00413), December 2023. arXiv:2312.00413 [cs]. URL: <http://arxiv.org/abs/2312.00413>, doi:10.48550/arXiv.2312.00413.

- [6] Jian Zhang, Xu Wang, Hongyu Zhang, Hailong Sun, Kaixuan Wang, and Xudong Liu. A novel neural source code representation based on abstract syntax tree. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*, page 783–794, May 2019. URL: <https://ieeexplore.ieee.org/document/8812062>, doi:10.1109/ICSE.2019.00086.
- [7] Dawei Yuan, Sen Fang, Tao Zhang, Zhou Xu, and Xiapu Luo. Java code clone detection by exploiting semantic and syntax information from intermediate code-based graph. *IEEE Transactions on Reliability*, 72(2):511–526, 2023. doi:10.1109/TR.2022.3176922.
- [8] Wenhan Wang, Ge Li, Bo Ma, Xin Xia, and Zhi Jin. Detecting code clones with graph neural network and flow-augmented abstract syntax tree. In *2020 IEEE 27th International Conference on Software Analysis, Evolution and Reengineering (SANER)*, page 261–271, February 2020. URL: <https://ieeexplore.ieee.org/document/9054857>, doi:10.1109/SANER48275.2020.9054857.
- [9] Qing Mi, Yi Zhan, Han Weng, Qinghang Bao, Longjie Cui, and Wei Ma. A graph-based code representation method to improve code readability classification. *Empirical Software Engineering*, 28(4):87, May 2023. doi:10.1007/s10664-023-10319-6.
- [10] Wei Hung Pan, Ming Jie Chok, Jonathan Leong Shan Wong, Yung Xin Shin, Yeong Shian Poon, Zhou Yang, Chun Yong Chong, David Lo, and Mei Kuan Lim. Assessing ai detectors in identifying ai-generated code: Implications for education. (arXiv:2401.03676), January 2024. arXiv:2401.03676 [cs]. URL: <http://arxiv.org/abs/2401.03676>, doi:10.48550/arXiv.2401.03676.
- [11] GPTZero. AI Detector - Most Accurate AI Checker for ChatGPT & Gemini. <https://gptzero.me>, 2023. Accessed: 2025-09-09.
- [12] Sapling. AI Detector | ChatGPT, Claude, Gemini. <https://sapling.ai/ai-content-detector>, 2023. Accessed: 2025-09-09.
- [13] Chip Thien. <https://github.com/MacroChip/gpt-2-output-dataset>, 2023. Accessed: 2025-09-09.
- [14] Eric Mitchell, Yoonho Lee, Alexander Khazatsky, Christopher D Manning, and Chelsea Finn. DetectGPT: Zero-shot machine-generated text detection using probability curvature. In Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett, editors, *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 24950–24962. PMLR, 23–29 Jul 2023. URL: <https://proceedings.mlr.press/v202/mitchell123a.html>.
- [15] Sebastian Gehrmann, Hendrik Strobelt, and Alexander M. Rush. Gltr: Statistical detection and visualization of generated text. (arXiv:1906.04043), june 2019. arXiv:1906.04043 [cs]. URL: <http://arxiv.org/abs/1906.04043>, doi:10.48550/arXiv.1906.04043.
- [16] Xiaodan Xu, Chao Ni, Xinrong Guo, Shaoxuan Liu, Xiaoya Wang, Kui Liu, and Xiaohu Yang. Distinguishing llm-generated from human-written code by contrastive learning. (arXiv:2411.04704), November 2024. arXiv:2411.04704 [cs]. URL: <http://arxiv.org/abs/2411.04704>, doi:10.48550/arXiv.2411.04704.

- [17] Daya Guo, Shuai Lu, Nan Duan, Yanlin Wang, Ming Zhou, and Jian Yin. Unixcoder: Unified cross-modal pre-training for code representation. (arXiv:2203.03850), March 2022. arXiv:2203.03850 [cs]. URL: <http://arxiv.org/abs/2203.03850>, doi:10.48550/arXiv.2203.03850.
- [18] Basak Demirok and Mucahid Kutlu. Aigcodeset: A new annotated dataset for ai generated code detection. (arXiv:2412.16594), May 2025. arXiv:2412.16594 [cs]. URL: <http://arxiv.org/abs/2412.16594>, doi:10.48550/arXiv.2412.16594.
- [19] Daniil Orel, Dilshod Azizov, and Preslav Nakov. Codet-m4: Detecting machine-generated code in multi-lingual, multi-generator and multi-domain settings. (arXiv:2503.13733), july 2025. arXiv:2503.13733 [cs]. URL: <http://arxiv.org/abs/2503.13733>, doi:10.48550/arXiv.2503.13733.
- [20] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. Codebert: A pre-trained model for programming and natural languages. (arXiv:2002.08155), sept 2020. arXiv:2002.08155 [cs]. URL: <http://arxiv.org/abs/2002.08155>, doi:10.48550/arXiv.2002.08155.
- [21] Yue Wang, Weishi Wang, Shafiq Joty, and Steven C.H. Hoi. Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. In Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih, editors, *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, page 8696–8708, Online and Punta Cana, Dominican Republic, nov 2021. Association for Computational Linguistics. URL: <https://aclanthology.org/2021.emnlp-main.685/>, doi:10.18653/v1/2021.emnlp-main.685.
- [22] Max Brunsfeld, Amaan Qureshi, Andrew Hlynskyi, ObserverOfTime, Will Lillis, Josh Vera, dundargoc, Phil Turnbull, Timothy Clem, Douglas Creager, Andrew Helwer, Rob Rix, Daumantas Kavolis, Hendrik van Antwerpen, Christian Clason, Michael Davis, Riley Bruins, Antonin Delpeuch, Ika, Amin Ya, Tun-Anh Nguyn, bfredl, Matt Massicotte, Stafford Brunk, Niranjan Hasabnis, James McCoy, Mingkai Dong, Samuel Moelius, and Steven Kalt. tree-sitter/tree-sitter: v0.25.9, September 2025. doi: 10.5281/zenodo.17069743.
- [23] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [24] Matthias Fey and Jan E. Lenssen. Fast graph representation learning with PyTorch Geometric. In *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.
- [25] Matthias Fey, Jinu Sunil, Akihiro Nitta, Rishi Puri, Manan Shah, Blaž Stojanovič, Ramona Bendias, Alexandria Barghi, Vid Kocijan, Zecheng Zhang, Xinwei He, Jan E. Lenssen, and Jure Leskovec. PyG 2.0: Scalable learning on real world graphs. In *Temporal Graph Learning Workshop @ KDD*, 2025.
- [26] Jason Ansel, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, Peter Bell, David Berard, Evgeni Burovski, Geeta Chauhan, Anjali Chourdia, Will Constable, Alban Desmaison, Zachary DeVito, Elias Ellison,

- Will Feng, Jiong Gong, Michael Gschwind, Brian Hirsh, Sherlock Huang, Kshiteej Kalambarkar, Laurent Kirsch, Michael Lazos, Mario Lezcano, Yanbo Liang, Jason Liang, Yinghai Lu, CK Luk, Bert Maher, Yunjie Pan, Christian Puhersch, Matthias Reso, Mark Saroufim, Marcos Yukio Siraichi, Helen Suk, Michael Suo, Phil Tillet, Eikan Wang, Xiaodong Wang, William Wen, Shunting Zhang, Xu Zhao, Keren Zhou, Richard Zou, Ajit Mathews, Gregory Chanan, Peng Wu, and Soumith Chintala. PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation. In *29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '24)*. ACM, April 2024. URL: <https://docs.pytorch.org/assets/pytorch2-2.pdf>, doi:10.1145/3620665.3640366.
- [27] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. (arXiv:1412.6980), jan 2017. arXiv:1412.6980 [cs]. URL: <http://arxiv.org/abs/1412.6980>, doi:10.48550/arXiv.1412.6980.
- [28] Jeffrey Pennington, Richard Socher, and Christopher D. Manning. Glove: Global vectors for word representation. In *Empirical Methods in Natural Language Processing (EMNLP)*, page 1532–1543, 2014. URL: <http://www.aclweb.org/anthology/D14-1162>.
- [29] Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. (arXiv:1609.02907), feb 2017. arXiv:1609.02907 [cs]. URL: <http://arxiv.org/abs/1609.02907>, doi:10.48550/arXiv.1609.02907.
- [30] William L. Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large graphs. (arXiv:1706.02216), sep 2018. arXiv:1706.02216 [cs]. URL: <http://arxiv.org/abs/1706.02216>, doi:10.48550/arXiv.1706.02216.
- [31] Yujia Li, Chenjie Gu, Thomas Dullien, Oriol Vinyals, and Pushmeet Kohli. Graph matching networks for learning the similarity of graph structured objects. (arXiv:1904.12787), May 2019. arXiv:1904.12787 [cs]. URL: <http://arxiv.org/abs/1904.12787>, doi:10.48550/arXiv.1904.12787.
- [32] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. (arXiv:1710.10903), feb 2018. arXiv:1710.10903 [stat]. URL: <http://arxiv.org/abs/1710.10903>, doi:10.48550/arXiv.1710.10903.
- [33] Yunsheng Shi, Zhengjie Huang, Shikun Feng, Hui Zhong, Wenjin Wang, and Yu Sun. Masked label prediction: Unified message passing model for semi-supervised classification. (arXiv:2009.03509), may 2021. arXiv:2009.03509 [cs]. URL: <http://arxiv.org/abs/2009.03509>, doi:10.48550/arXiv.2009.03509.
- [34] Zixian Zhang and Takfarinas Saber. Ast-enhanced or ast-overloaded? the surprising impact of hybrid graph representations on code clone detection. (arXiv:2506.14470), 2025. arXiv:2506.14470 [cs]. URL: <http://arxiv.org/abs/2506.14470>, doi:10.48550/arXiv.2506.14470.